

Atividade 18 – Trigger e Procedures

Lista de Exercícios – Procedures e Triggers no PostgreSQL

Parte I – Reconhecimento do banco

Exercício 1

Liste todas as tabelas existentes no banco e inspecione a estrutura de reserva, hospede, quarto e tipo_quarto.

Resposta:

- - Listar todas as tabelas

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
```

- - Inspeccionar estruturas

```
\d reserva
\d hospede
\d quarto
\d tipo_quarto
```

Exercício 2

Conte quantos registros existem nas tabelas tipo_quarto, quarto, hospede e reserva.

Resposta:

```
SELECT
    (SELECT COUNT(*) FROM tipo_quarto) AS total_tipo_quarto,
    (SELECT COUNT(*) FROM quarto) AS total_quarto,
    (SELECT COUNT(*) FROM hospede) AS total_hospede,
    (SELECT COUNT(*) FROM reserva) AS total_reserva;
```

Exercício 3

Liste as procedures e funções existentes no banco. Depois, visualize o código de sp_criar_reserva e fn_validar_reserva.

Resposta:

- - Listar procedures e funções

```
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'public';
```

- - Visualizar código de sp_criar_reserva

```
\sf sp_criar_reserva
```

- - Visualizar o código de fn_validar_reserva

```
\sf fn_validar_reserva
```

Parte II – Procedures existentes

Exercício 4

Execute `sp_reajustar_diarias` para aumentar todas as diárias em 4%. Consulte `tipo_quarto` antes e depois.

Resposta:

- - Consulta antes do reajuste

```
SELECT * FROM tipo_quarto;
```

- - Reajuste de 4% em todas as diárias

```
CALL sp_reajustar_diarias(4);
```

- - Consulta depois do reajuste

```
SELECT * FROM tipo_quarto;
```

Exercício 5

Execute `sp_reajustar_diarias` para alterar apenas um tipo de quarto, escolhido pelo `id_tipo`.

Resposta:

- - Reajuste de 5% apenas para o `tipo_quarto id = 1`

```
CALL sp_reajustar_diarias(5, 1);
```

```
SELECT * FROM tipo_quarto WHERE id_tipo = 1;
```

Exercício 6

Utilize `sp_criar_reserva` para criar uma reserva válida. Depois consulte `id`, `hóspede`, `quarto`, `check-in`, `checkout`, `quantidade de hóspedes` e `valor total`.

Resposta:

- - Criar reserva válida (Hóspede 1, Quarto 1 [capacidade 2], período válido)

```
CALL sp_criar_reserva(1, 1, '2026-12-01', '2026-12-05', 2);
```

- - Consultar a reserva criada

```
SELECT id_reserva, id_hospede, id_quarto, data_checkin, data_checkout,  
quantidade_hospedes, valor_total  
FROM reserva  
ORDER BY id_reserva DESC  
LIMIT 1;
```

Exercício 7

Tente criar uma reserva com `checkout` anterior ao `check-in`. Anote a mensagem de erro e explique por que a operação foi bloqueada.

Resposta:

- - Tentativa de reserva com `checkout` anterior ao `check-in`

```
CALL sp_criar_reserva(1, 1, '2026-12-10', '2026-12-05', 1);
```

Mensagem de erro: ERROR: A data de checkout deve ser posterior ao check-in.

Explicação: A operação é bloqueada pela trigger trg_validar_reserva (função fn_validar_reserva), que valida as datas antes de inserir e cancela a transação disparando um RAISE EXCEPTION. Além disso, a própria tabela reserva possui uma restrição CHEK (data_checkout > data_checkin).

Exercício 8

Tente criar uma reserva com quantidade de hóspedes maior que a capacidade do quarto. Antes, consulte número, tipo e capacidade do quarto escolhido.

Resposta:

- - Consulta de verificação (Quarto 1 = Tipo Standard = Capacidade 2)

```
SELECT q.id_quarto, q.numero, tq.nome_tipo, tq.capacidade
FROM quarto q
JOIN tipo_quarto tq ON q.id_tipo = tq.id_tipo
WHERE q.id_quarto = 1;
```

- - Tentativa de criar reserva para 10 pessoas em um quarto de capacidade 2

```
CALL sp_criar_reserva(1, 1, '2026-12-01', '2026-12-05', 10);
```

Parte III – Criando Procedures

Exercício 9

Crie uma procedure sp_marcar_notificacao_processada que receba id_notificacao, altere processada para TRUE e lance exceção caso a notificação não exista.

Resposta:

```
CREATE OR REPLACE PROCEDURE sp_marcar_notificacao_processada(
    p_id_notificacao BIGINT
)
LANGUAGE plpgsql
AS $$

BEGIN
    UPDATE notificacao
    SET processada = TRUE
    WHERE id_notificacao = p_id_notificacao;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Notificação com ID % não encontrada.',
            p_id_notificacao;
    END IF;
    RAISE NOTICE 'Notificação % marcada como processada.', p_id_notificacao;

END;
```

\$\$

Exercício 10

Crie uma procedure sp_colocar_quarto_manutencao que receba id_quarto, altere o status para Manutenção, use RAISE NOTICE e lance exceção se o quarto não existir.

Resposta:

```
CREATE OR REPLACE PROCEDURE sp_colocar_quarto_manutencao(  
    p_id_quarto INTEGER  
)  
  
LANGUAGE plpgsql  
AS $$  
  
BEGIN  
    UPDATE quarto  
    SET status = 'Manutenção'  
    WHERE id_quarto = p_id_quarto;  
  
    IF NOT FOUND THEN  
        RAISE EXCEPTION 'Quarto com ID % não encontrado.', p_id_quarto;  
    END IF;  
  
    RAISE NOTICE 'Quarto % alterado para status Manutenção.', p_id_quarto;  
  
END;  
$$
```

Exercício 11

Crie uma procedure sp_liberar_quarto que receba id_quarto, altere o status para Disponível e lance exceção se o quarto não existir.

Resposta:

```
CREATE OR REPLACE PROCEDURE sp_liberar_quarto(  
    p_id_quarto INTEGER  
)  
  
LANGUAGE plpgsql  
AS $$  
  
BEGIN  
    UPDATE quarto  
    SET status = 'Disponível'  
    WHERE id_quarto = p_id_quarto;  
  
    IF NOT FOUND THEN  
        RAISE EXCEPTION 'Quarto com ID % não encontrado.', p_id_quarto;  
    END IF;
```

```

        RAISE NOTICE 'Quarto % alterado para status Disponível.', p_id_quarto;

END;
$$

```

Exercício 12

Crie uma procedure `sp_alterar_status_reserva` que receba o id da reserva e um novo status. Permita apenas Confirmada, Hospedada, Concluída e Cancelada. Para qualquer outro valor, use `RAISE EXCEPTION`.

Resposta:

```

CREATE OR REPLACE PROCEDURE sp_alterar_status_reserva(
    p_id_reserva BIGINT,
    p_novo_status VARCHAR(20)
)

LANGUAGE plpgsql
AS $$

BEGIN
    IF p_novo_status NOT IN ('Confirmada', 'Hospedada', 'Cancelada') THEN
        RAISE EXCEPTION 'Status inválido: %. Permitidos: Confirmada, Hospedada, Concluída, Cancelada.', p_novo_status;
    END IF;

    UPDATE reserva
    SET status = p_novo_status
    WHERE id_reserva = p_id_reserva;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Reserva com ID % não encontrada.', p_id_reserva;
    END IF;

    RAISE NOTICE 'Status da reserva % alterado para %.', p_id_reserva,
    p_novo_status;

END;
$$

```

Exercício 13

Crie uma procedure `sp_aplicar_desconto_reserva` que receba id da reserva e percentual de desconto. Valide o percentual entre 0 e 100, reduza `valor_total` e mostre o novo valor com `RAISE NOTICE`.

Resposta:

```

CREATE OR REPLACE PROCEDURE sp_aplicar_desconto_reserva(
    p_id_reserva BIGINT,

```

```

        p_percentual NUMERIC
    )

LANGUAGE plpgsql
AS $$

DECLARE
    v_novo_valor NUMERIC(10, 2);

BEGIN
    IF p_percentual < 0 OR p_percentual > 100 THEN
        RAISE EXCEPTION 'Percentual de desconto inválido (%). Deve estar entre 0
        e 100.', p_percentual;
    END IF;

    UPDATE reserva
    SET valor_total = ROUND(valor_total * (1 - p_percentual/100), 2)
    WHERE id_reserva = p_id_reserva
    RETURNING valor_total INTO v_novo_valor;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Reserva com ID % não encontrada.', p_id_reserva;
    END IF;

    RAISE NOTICE 'Desconto aplicado com sucesso. Novo valor total da reserva %: R$
    %' p_id_reserva, v_novo_valor;

END;
$$

```

Parte IV – Criando Triggers de Validação

Exercício 14

Crie a função fn_validar_capacidade_quarto. Ela deve impedir INSERT ou UPDATE em reserva quando quantidade_hospedes exceder a capacidade do tipo de quarto.

A função deve consultar a capacidade, comparar com NEW.quantidade_hospedes, usar RAISE EXCEPTION em caso inválido e retornar NEW quando válido.

Crie a trigger trg_validar_capacidade_quarto como BEFORE INSERT OR UPDATE em reserva.

Resposta:

```
CREATE OR REPLACE FUNCTION fn_validar_capacidade_quarto()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
AS $$
```

```

DECLARE
    v_capacidade INTEGER;

BEGIN
    SELECT tq.capacidade INTO v_capacidade
    FROM quarto q
    JOIN tipo_quarto tq ON q.id_quarto = tq.id_tipo
    WHERE q.id_quarto = NEW.id_quarto;

    IF NEW.quantidade_hospedes > v_capacidade THEN
        RAISES EXCEPTION 'A quantidade de hóspedes (%) excede a capacidade
        do quarto (%)', NEW.quantidade_hospedes, v_capacidade;
    END IF;

    RETURN NEW;

END;
$$

CREATE TRIGGER trg_validar_capacidade_quarto
BEFORE INSERT OR UPDATE ON reserva
FOR EACH ROW
EXECUTE FUNCTION fn_validar_capacidade_quarto();

```

Exercício 15

Crie fn_validar_quarto_disponivel para impedir uma nova reserva quando o quarto estiver Manutenção ou Inativo.

Crie trg_validar_quarto_disponivel como BEFORE INSERT em reserva.

Teste colocando um quarto em manutenção com a procedure do Exercício 10.

Resposta:

```

CREATE OR REPLACE FUNCTION fn_validar_quarto_disponivel()

```

```

RETURNS TRIGGER

```

```

LANGUAGE plpgsql
AS $$

```

```

DECLARE
    v_status VARCHAR(20);

```

```

BEGIN
    SELECT status INTO v_status
    FROM quarto
    WHERE id_quarto = NEW.id_quarto;

    IF v_status IN ('Manutenção', 'Inativo') THEN

```

```

        RAISE EXCEPTION 'Não é possível reservar o quarto %. Status atual: %.',
        NEW.id_quarto, v_status);
    END IF;

    RETURN NEW;

END;
$$

```

```

CREATE TRIGGER trg_validar_quarto_disponivel
BEFORE INSERT ON reserva
FOR EACH ROW
EXECUTE FUNCTION fn_validar_quarto_disponivel();

```

-- Teste:

-- 1. Colocar o quarto 1 em Manutenção

```
CALL sp_colocar_quarto_manutencao(1);
```

-- 2. Tenta fazer reserva no quarto 1 (deve falhar)

```
CALL sp_criar_reserva(1, 1, '2026-12-15', '2026-12-20', 1);
```

Exercício 16

Crie fn_validar_datas_reserva para impedir uma reserva quando date_checkout <= date_checkin.

Crie trg_validar_datas_reserva como BEFORE INSERT OR UPDATE em reserva.

Resposta:

```
CREATE OR REPLACE FUNCTION fn_validar_datas_reserva()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    IF NEW.data_checkout <= NEW.data_checkin THEN
```

```
        RAISE EXCEPTION 'Data de checkout (%) deve ser estritamente maior que a
        data check-in (%).', NEW.data_checkout, NEW.data_checkin;
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$
```

```

CREATE TRIGGER trg_validar_datas_reserva
BEFORE INSERT OR UPDATE ON reserva
FOR EACH ROW

```

EXECUTE FUNCTION fn_validar_datas_reserva();

Parte V – Criando Triggers de Auditoria

Exercício 17

Crie a tabela auditoria_status_reserva com:

- id_auditoria
- id_reserva
- status_anterior
- status_novo
- data_alteracao

Crie fn_auditar_status_reserva usando OLD.status e NEW.status, registrando apenas mudanças reais de status.

Resposta:

- - Criação da tabela auditoria_status_reserva

```
CREATE TABLE auditoria_status_reserva (  
    id_auditoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_reserva BIGINT NOT NULL,  
    status_anterior VARCHAR (20),  
    status_novo VARCHAR (20),  
    data_alteracao TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- - Criação do trigger

```
CREATE OR REPLACE FUNCTION fn_auditar_status_reserva()
```

```
RETURNS TRIGGERS
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    IF OLD.status IS DISTINCT FROM NEW.status THEN
```

```
        INSERT INTO auditoria_status_reserva (id_reserva, status_anterior,  
        status_novo)
```

```
        VALUES (NEW.id_reserva, OLD.status, NEW.status);
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$
```

```
CREATE TRIGGER trg_auditar_status_reserva
```

```
AFTER UPDATE OF status ON reserva
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION fn_auditar_status_reserva();
```

Exercício 18

Teste a trigger anterior alterando o status de uma reserva. Depois consulte auditoria_status_reserva.

Resposta:

-- Teste de alteração de status

```
UPDATE reserva
```

```
SET status = 'Hospedada'
```

```
WHERE id_reserva = 1;
```

-- Consulta da tabela de auditoria

```
SELECT * FROM auditoria_status_reserva;
```

Exercício 19

Crie uma tabela auditoria_valor_diaria com id da auditoria, id do tipo, valor anterior, valor novo e data da alteração.

Crie uma função e uma trigger para registrar alterações em tipo_quarto.valor_diaria.

A trigger deve executar apenas após atualização de valor_diaria.

Resposta:

-- Criação da tabela auditoria_valor_diaria

```
CREATE TABLE auditoria_valor_diaria (  
    id_auditoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_tipo INTEGER NOT NULL,  
    valor_anterior NUMERIC(10, 2),  
    valor_novo NUMERIC(10, 2),  
    data_alteracao TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Criação do trigger

```
CREATE OR REPLACE FUNCTION fn_auditar_valor_diaria()
```

```
RETURNS TRIGGERS
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    IF OLD.valor_diaria IS DISTINCT FROM NEW.valor_diaria THEN  
        INSERT INTO auditoria_valor_diaria (id_tipo, valor_anterior,  
            valor_novo)  
            VALUES (NEW.id_tipo, OLD.valor_diaria, NEW.valor_diaria);  
    END IF;
```

```
    RETURN NEW;
```

```
END;  
$$
```

```
CREATE TRIGGER trg_auditar_valor_diaria  
AFTER UPDATE OF valor_diaria ON tipo_quarto  
FOR EACH ROW  
EXECUTE FUNCTION fn_auditar_valor_diaria();
```

Parte VI – Triggers para INSERT e DELETE sem TG_OP

Exercício 20

Crie fn_auditar_nova_reserva para registrar, após um INSERT em reserva:

- id da reserva
- id do hóspede
- id do quarto
- valor total
- data da criação

Crie trg_auditar_nova_reserva como AFTER INSERT.
Use somente NEW.

Resposta:

- - Criação da tabela auditoria_nova_reserva

```
CREATE TABLE auditoria_nova_reserva (  
    id_auditoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_reserva BIGINT,  
    id_hospede INTEGER,  
    id_quarto INTEGER,  
    valor_total NUMERIC(10, 2),  
    data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- - Criação do trigger

```
CREATE OR REPLACE FUNCTION fn_auditar_nova_reserva()
```

```
RETURNS TRIGGERS
```

```
LANGUAGE plpgsql  
AS $$
```

```
BEGIN
```

```
    INSERT INTO auditoria_nova_reserva (id_reserva, id_hospede, id_quarto,  
    valor_total)  
    VALUES (NEW.id_reserva, NEW.id_hospede, NEW.id_quarto, NEW.valor_total);
```

```
    RETURN NEW;
```

```
END;  
$$
```

```
CREATE TRIGGER trg_auditar_nova_reserva  
AFTER INSERT ON reserva  
FOR EACH ROW  
EXECUTE FUNCTION fn_auditar_nova_reserva();
```

Exercício 21

Crie uma função separada fn_auditar_exclusao_reserva para registrar, após DELETE:

- id da reserva excluída
- status anterior
- valor anterior
- data da exclusão

Crie trg_auditar_exclusao_reserva como AFTER DELETE.
Use somente OLD.

Não use a mesma função dos exercícios de INSERT e DELETE.

Resposta:

- Criação da tabela auditoria_exclusao_reserva

```
CREATE TABLE auditoria_exclusao_reserva (  
    id_auditoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_reserva BIGINT,  
    status_anterior VARCHAR(20),  
    valor_anterior NUMERIC(10, 2),  
    data_alteracao TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- Criação do trigger

```
CREATE OR REPLACE FUNCTION fn_auditar_exclusao_reserva()
```

```
RETURNS TRIGGERS
```

```
LANGUAGE plpgsql  
AS $$
```

```
BEGIN
```

```
    INSERT INTO auditoria_exclusao_reserva (id_reserva, status_anterior,  
        valor_anterior)  
    VALUES (OLD.id_reserva, OLD.status, OLD.valor_total);
```

```
    RETURN OLD;
```

```
END;  
$$
```

```
CREATE TRIGGER trg_auditar_exclusao_reserva
AFTER DELETE ON reserva
FOR EACH ROW
EXECUTE FUNCTION fn_auditar_exclusao_reserva();
```

Parte VI – Triggers Condicional com WHEN

Exercício 22

Crie fn_notificar_reserva_alto_valor para inserir uma notificação.
Crie trg_notificar_reserva_alto_valor como AFTER INSERT somente quando NEW.valor_total > 2000, usando WHEN

Resposta:

```
CREATE OR REPLACE FUNCTION fn_notificar_reserva_alto_valor()

RETURNS TRIGGER

LANGUAGE plpgsql
AS $$

BEGIN
    INSERT INTO notificacao (id_reserva, mensagem)
    VALUES (NEW.id_reserva, 'Reserva de alto valor registrada! ID: ' || NEW.id_reserva
    || ' | Valor: R$ ' || NEW.valor_total);

    RETURN NEW;

END;
$$

CREATE TRIGGER trg_notificar_reserva_alto_valor
AFTER INSERT ON reserva
FOR EACH ROW
WHEN (NEW.valor_total > 2000)
EXECUTE FUNCTION fn_notificar_reserva_alto_valor();
```

Exercício 23

Crie fn_notificar_cancelamento_aluno para inserir uma notificação quando uma reserva mudar para Cancelada.
Crie trg_notificar_cancelamento_aluno como AFTER UPDATE OF status usando WHEN com OLD.status e NEW.status.

Resposta:

```
CREATE OR REPLACE FUNCTION fn_notificar_cancelamento_aluno()

RETURNS TRIGGER
```

```
LANGUAGE plpgsql
AS $$
```

```
BEGIN
    INSERT INTO notificacao (id_reserva, mensagem)
    VALUES (NEW.id_reserva, 'Atenção: A reserva ' || NEW.id_reserva || ' mudou para
    Cancelada.');
```

```
    RETURN NEW;
```

```
END;
$$
```

```
CREATE TRIGGER trg_notificar_cancelamento_aluno
AFTER UPDATE OF status ON reserva
FOR EACH ROW
WHEN (OLD.status <> 'Cancelada' AND NEW.status = 'Cancelada')
EXECUTE FUNCTION fn_notificar_cancelamento_aluno();
```